作业 1：搜索

清华大学软件学院
人工智能导论, 2026 年秋季学期

介绍

本次作业需要提交说明文档（PDF 形式）和 Python 的源代码。注意事项如下：

- 本次作业满分为 100 分，附加题 5 分，得分超过 100 分按 100 分记。
- 除简答题、编程题外的题目，请给出必要的解答过程，只有答案且过于简略的回答会酌情扣除一定分数。题目要求直接回答或只汇报结果的题目，则不需要给出过程或分析。
- **请不要使用他人的作业，也不要向他人公开自己的作业，复制网上内容须在报告中说明**，否则将受到严厉处罚，作业分数扣至-100（即倒扣本次作业的全部分值）。
- 完成作业过程中，如果使用了大模型辅助（如润色文笔、询问知识点等），请在作业末尾声明使用的方式和程度（不影响作业评分）。**禁止直接粘贴大模型输出的文本**，否则会扣除一定的作业分数。
- 统一文件的命名：`{学号}_{姓名}_hw1.zip`。**所有解答和实验报告请写在一个 pdf 文件中**，和代码一起压缩上传。

1 A* 算法的性质（10 分）

在 A* 算法中，记搜索起点为 S ，终点为 T。对于节点 n，记 $g(n)$ 表示当前搜索过程中从根节点到 n 的总代价，$h^*(n)$ 为 n 到目标节点 T 的最小代价，$h(n)$ 为 n 节点的启发 (heuristic) 函数。在以下问题中，设 $h(T) = 0$，且所有代价均非负。假设 A* 算法每次选择边缘集里 $f(n) = g(n) + h(n)$ 最小的节点。若采用树搜索：

1. 证明：若对任意节点 n 均满足 $h(n) \leq h^*(n) + C1$，其中 $C1 \geq 0$ 为常数，则 $g(T) \leq h^*(S) + C1$；
2. 证明：若对任意节点 n 均满足 $h(n) \leq C2 \cdot h^*(n)$，其中 $C2 \geq 1$ 为常数，则 $g(T) \leq C2 \cdot h^*(S)$。
3. (开放式问题）如果是图搜索，会有类似的结论吗?

提示：算法**未必**会找到 S 到 T 的最优路径。讨论当 T 即将取出边缘集时，最优路径上位于边缘集中的点 n^*。事实上，n^* 满足 $h^*(S) = g(n^*) + h^*(n^*)$

2 计算题（35 分）

（本题无编程，借助代码给大家更美观的界面）根据 README 的要求配置环境后，运行以下命令即可启动可视化程序：`python code/calculation/visualizer.py`

请使用自己的学号生成题目（学号为随机种子，不同学号生成题目不一样），完成作答后将答案正确的截图贴在报告里（允许修改答案，可以多次提交）。

本题考察 BFS、A* 等搜索算法的过程，几种局部搜索算法的过程。Minimax 和 Alpha-beta 题目不计入本次作业成绩，仅供练习使用。



程序同样支持随机生成题目，期末复习时也欢迎大家使用此程序练习相关知识点。

3 数独问题与搜索算法（15 分）

本题为编程题，代码和相关文档在`./code/sudoku`目录下。

请分别使用回溯搜索（Backtracking Search）和局部搜索（Local Search）求解 4×4 数独问题：在一个空的 4×4 棋盘中填入数字 1–4，使得每一行、每一列以及每个 2×2 小宫格内均包含数字 1–4，且不能重复。本题不提供任何初始数字。棋盘使用二维数组 *board* 表示，其中 $board[i][j]$ 表

示第 i 行、第 j 列格子中的数字；在回溯搜索中，数字 0 表示该格子尚未填写。

1. 请补全 `backtracking.py` 中的 `backtrack` 函数，使用回溯搜索枚举所有满足约束的数独解。
2. 请补全 `local_search.py` 中的 `get_conflicts` 函数和 `min_conflicts_steps` 函数，使用最小冲突启发式（Min-Conflicts Heuristic）求解数独。局部搜索开始时，应随机初始化棋盘，并保证每一行均为数字 1–4 的一个随机排列；之后每次随机选取一个存在冲突的格子，分别尝试将其与同一行中的其他三个数字交换，并在总冲突数最小的候选交换中随机选择一个执行。搜索过程中唯一允许的状态修改操作是交换同一行中的两个数字。
3. 简答题：还有哪些技术可以求解数独问题？请给出思路并简述算法。（仅举例，同样欢迎其他角度的讨论：比如可以围绕课上介绍的遗传算法等展开分析）

一个格子的冲突数是指：在同一列或同一个 2×2 小宫格中，与该格子数字相同的其他格子数量；同一对冲突只计一次。若达到最大搜索步数仍未找到解，则重新随机初始化棋盘并继续搜索。

可视化：在项目根目录运行 `python3 code/sudoku/visualizer.py` 程序将自动在浏览器中打开可视化界面。



4 蒙特卡洛树搜索与 Alpha–Beta 搜索（40 分）

本题为编程题，代码和相关文档在下发文件的 `./code/mcts` 目录下。

问题背景 围棋（Go）是一种常见的棋类对弈游戏，对局双方轮流在棋盘上的合法位置落子。本题只考察简化围棋环境，主要使用 5×5 和 7×7 棋盘。详细规则请参阅 `doc/AI2026_HW_GoEnvManual.pdf`。你只需了解代码手册中的合法动作、停一手、终局判定和奖励定义，不需要精通围棋知识。

任务目标 在本题中，你将实现基于 UCT 的蒙特卡洛树搜索（MCTS），并完成一个带启发式局面评估的深度限制 Alpha–Beta 搜索。你将通过可视化界面观察搜索树的构建过程，并比较 Random、MCTS、启发式 MCTS 和 Alpha–Beta 在围棋上的速度与棋力。

请根据要求使用和修改 `code/mcts` 路径下的代码，提交代码和实验报告。提交时请删除 `*.so`、`*.pyd` 和 `*/build/` 等编译或临时文件，本题的文字报告请和其他题目写在同一份文档中提交。

1. **实现 MCTS**。补全 `mcts/uct_mcts.py` 中标注 `TODO` 的内容，完成 UCT 动作选择、节点扩

展、随机模拟、价值回传和根节点策略计算。具体价值视角、访问次数和合法动作的约定请以 `README.md` 为准。

2. **实现 Alpha–Beta**。补全 `other_algo/alpha_beta_search.py` 中标注的 3 处 `TODO`：Negamax 递归调用、`value/alpha` 更新和 Alpha–Beta 剪枝条件。局面评估函数和棋手封装已经提供，不需要自行实现。由于搜索开销较大，Alpha–Beta 只允许用于 5×5 围棋；建议使用深度 1 或 2 进行调试。
3. **对比基本棋手**。在 5×5 围棋上，设置 $C = 1.0$、`n_rollout`$= 7$、`n_search`$= 64$，Alpha–Beta 深度设为 1。分别运行 MCTS 对弈 Random 和 MCTS 对弈 Alpha–Beta，并让 MCTS 分别执黑、执白，共得到 4 组实验。每种执棋顺序至少运行 10 局，使用固定随机种子，汇报 MCTS 的胜率、不输率和总用时，并简要分析结果。
4. **分析探索常数 C**。分别设置 $C = 0.1$ 和 $C = 5.0$。利用可视化界面对比搜索树中各动作的访问次数与根节点策略，说明 C 如何影响探索与利用之间的平衡。
5. **分析 MCTS 的搜索预算**。自行选取一组合理的 `n_rollout` 和 `n_search`，在 7×7 围棋上测试 MCTS 对弈 Random。让 MCTS 分别执黑和执白，每种执棋顺序至少运行 10 局，汇报胜率、不输率和用时，并说明这两个参数对 MCTS 搜索速度和策略质量的影响。
6. **使用带评估函数的 MCTS**。当问题规模扩大时，随机模拟的时间开销较大。`other_algo/heuristic.py` 中提供了一个围棋局面评估函数，以及可直接调用的带评估函数的 MCTS 棋手 `HeuristicUCTPlayer`。请在固定棋盘大小、对手、随机种子和搜索预算的条件下，对比普通 MCTS 与启发式 MCTS 的搜索用时和对局结果，并简要分析使用局面评估替代或修正蒙特卡洛采样的优缺点。

可视化与实验　在项目根目录运行：`python code/mcts/visualizer.py` 程序会在浏览器中打开围棋可视化界面。单局模式可以逐事件查看 MCTS 的选择、扩展、模拟和回传，以及 Alpha–Beta 的节点展开、价值更新和剪枝。页面下方的“批量对局实验”可以选择 Random、MCTS、启发式 MCTS 或 Alpha–Beta，设置参数、固定随机种子并自动交换执棋顺序。`pit.py` 仍保留为可选的脚本化实验入口，仅在需要更大规模的自定义实验时使用。



提示

1. 动手之前，请仔细阅读 `README.md`，按照要求配置运行环境，并确保已了解问题定义和代码框架。请尽量不要修改代码中未要求修改的部分，如确有必要请提交前与助教沟通。
2. 本题中 `step()` 返回的终局 `reward` 属于刚刚落子的玩家；搜索后继必须使用 `fork()` 复制棋盘，不得直接修改搜索树节点保存的环境。
3. Alpha–Beta 中的“深度”指从根节点选择一次动作后，向下最多搜索的总落子层数。深度每增加 1，搜索量都会显著增大。

4. 做算法对比时，请固定棋盘大小、随机种子和除待比较参数以外的其他设置，并同时报告对局数与搜索用时。
5. 请确保代码具有良好的可读性与可复用性，后续作业会继续使用本次作业中的 MCTS 部分代码。

5 附加题：理解 AlphaZero（5 分）

在标准的围棋棋盘大小（19×19）下，由于状态空间极大，本次作业中使用的基于 UCB（Upper Confidence Bound）公式的 MCTS 无法在合理时间内得到高质量的策略。为了解决这个问题，AlphaZero 中，神经网络被嵌入 MCTS 以提高搜索的效率和质量。由于我们课程还未讲到神经网络，本次作业中的 MCTS 没有涉及到这部分内容。**为了更好的了解大作业的整体结构，确保在后续作业中可以零成本复用本次作业的代码**，请你简单阅读 AlphaGoZero 的论文[1]（其 MCTS 部分和 AlphaZero 的实现基本一致）和补充材料[2]，其中与本次作业内容较为相关的部分是 AlphaGoZero 论文的 "Reinforcement learning in AlphaGo Zero" 章节和补充材料第 14 至 15 页的内容。请在完成阅读后在实验报告中简要回答以下问题：

1. 为什么 AlphaZero 使用了 MCTS 而不是 AlphaBeta 作为搜索主干?
2. AlphaZero 中使用的基于 PUCT（Predictor + UCB Applied to Trees）的 MCTS 与本次作业中基于 UCT（Upper Confidence Bounds Applied to Trees）的 MCTS 有哪些区别?

[注]：你也可以参考网上的其它资料。

6 提交格式

- 请先删除 `*.so` `*.pyd` 等文件和环境目录下的 `build` 文件夹，再将你的代码目录内**所有代码文件**和你的**文字报告**打包提交。统一文件的命名：`{学号}_{姓名}_hw1.zip`。
- 请将本次作业所有问题回答写在同一份报告中，报告请导出为 **pdf 格式**。

[1] `https://www.nature.com/articles/nature24270`

[2] `https://www.science.org/doi/suppl/10.1126/science.aar6404/suppl_file/aar6404-silver-sm.pdf`